

Counterfeit Medicine Checker

Step-by-step implementation guide for our three-member team

github.com/Raviyaa27/counterfeit-medicine-checker

Prepared 26 September 2026

We're building a web app that lets anyone check whether a medicine pack is genuine by scanning its QR code with a phone. Every pack is registered on the Ethereum blockchain by an approved manufacturer, approved by the NMRA, tracked through the distributor to the pharmacy, and marked **sold** at the counter. A photocopied QR code then shows **already sold**, and a made-up code shows **not found**.

What we deliver	A working system on a public testnet, a 3-minute presentation with a demo video, and a clean GitHub repo.
Blockchain	Ethereum Sepolia testnet (free test ETH); local Anvil chain for development
Smart contract	Solidity 0.8.30 + OpenZeppelin 5.6.1 AccessControl, built and tested with Foundry
Web app	Next.js 16 + wagmi 3 / viem 2 , hosted free on Vercel
Repository	https://github.com/Raviyaa27/counterfeit-medicine-checker
Team	Member 1: blockchain lead · Member 2: frontend lead · Member 3: security, testing & presentation lead
Deadline	Brief says 31/09/2026, which doesn't exist. We plan to submit on Tuesday 29 September and treat Wednesday 30 September as a buffer.

The core code in this guide has already been built and tested

- The smart contract, deploy script and a local demo script: all 20 tests pass (unit, attack, fuzz and invariant).
- Slither static analysis: no medium or high findings, only 4 expected low-severity notes.
- The key frontend files: they type-check, build with Next.js 16, and the verify page correctly showed GENUINE, ALREADY SOLD and NOT FOUND against a local chain.

So your job is mostly to set up, understand, integrate, extend, deploy and present, not to start from a blank page. The full, tested code is in the appendices.

How to use this guide

1. **Everyone** reads Sections 1–4 together (about 30 minutes).
2. **Everyone** installs tools and creates accounts (Section 4) and starts collecting test ETH today, because faucets are slow.
3. **Member 1** sets up the repository (Section 5) and pushes it. The others then clone it.
4. Each member follows their own track: **Section 6** (Member 1), **Section 7** (Member 2), **Section 8** (Member 3).
5. Use the day-by-day plan in Section 2 for a short check-in each evening.

Contents

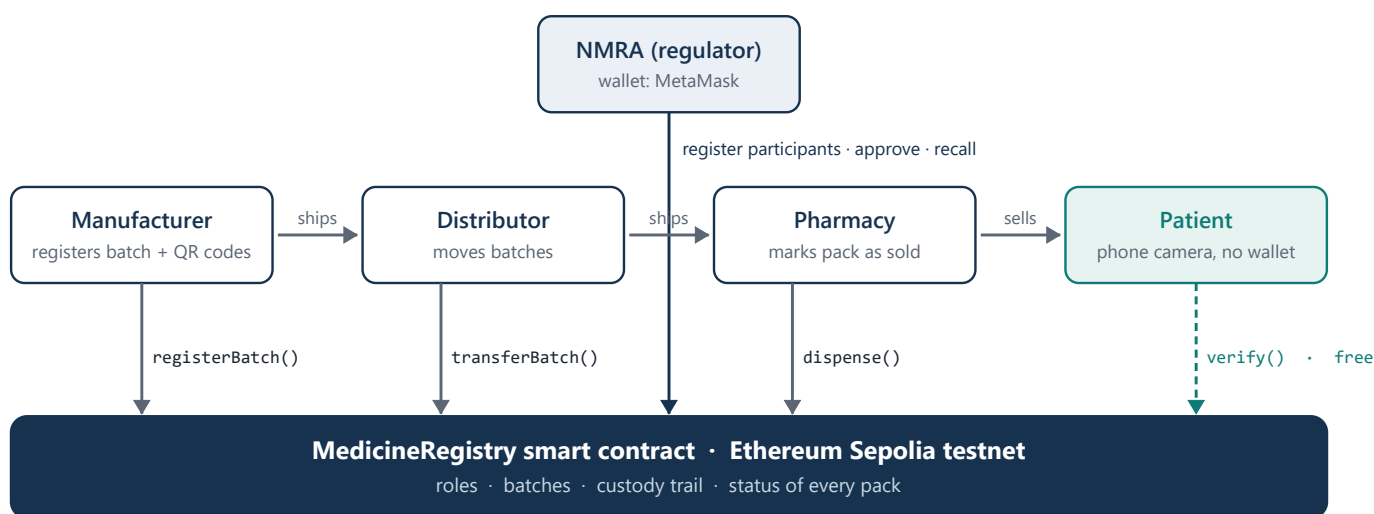
1 What we're building	3
2 Work division and timeline	4
3 Design decisions (everyone should understand these)	6
4 Tools and accounts (everyone, first two hours)	8
5 Repository setup (Member 1, once, about 30 minutes)	10
6 Member 1 track: blockchain lead	12
7 Member 2 track: frontend lead	15
8 Member 3 track: security, testing and presentation lead	17
9 Security: threat model	19
10 Demo and presentation	20
11 Troubleshooting	22
12 Stretch goals (only after the definition of done)	22
Appendix A Smart-contract files	23
Appendix B Test files	28
Appendix C Frontend files	33
Appendix D CI workflow and local demo script	40

1 What we're building

1.1 The problem

- The WHO estimates that about **1 in 10 medical products** in low- and middle-income countries is substandard or falsified (2017).
- In Sri Lanka, the **2023 substandard immunoglobulin scandal** showed how fake or substandard medicines can enter even the government supply chain.
- Today, manufacturers, the NMRA, distributors and pharmacies each keep separate records. A patient has no way to check a pack at all.

1.2 How the system works



1. The **NMRA** registers licensed manufacturers, distributors and pharmacies (each is a wallet address with a role).
2. A **manufacturer** registers a batch. Each pack gets a secret random serial, printed as a QR code. Only a *hash* of each serial goes on-chain.
3. The NMRA **approves** the batch. It then moves manufacturer → distributor → pharmacy, and every handover is recorded.
4. At the counter, the **pharmacy** scans the pack and marks it **sold**. This works only once, and only for the pharmacy that holds the batch.
5. A **patient** scans the QR code with their phone camera. The web page reads the blockchain (free, no wallet) and shows GENUINE, ALREADY SOLD, NOT FOUND, RECALLED or EXPIRED.

1.3 Why blockchain, and not a normal database?

- Several organisations that don't fully trust each other need to write to the same record. With a database, whoever runs it can quietly edit or delete entries.
- Records can't be changed after the fact. Every approval, handover and sale is time-stamped and public.
- Anyone can verify a pack without an account or permission from a central operator.
- Access rules (who can register, approve, ship or sell) are enforced by code, not by trust.

Real-world precedent: the EU Falsified Medicines Directive

Since 2019, prescription medicines in the EU carry a unique code, and pharmacies "decommission" it when they dispense the pack, so a copied code is detected. Our design follows the same idea, but on a shared ledger that no single party controls. It's a strong point to mention in the presentation.

2 Work division and timeline

Each member owns a separate part of the repository, so we rarely edit the same files. Write your name in your column.

Member 1 Blockchain lead	Member 2 Frontend lead	Member 3 Security, testing & presentation lead
Name: _____ Owns - contracts/src, contracts/script - Local and Sepolia deployments - Frontend: /admin (NMRA) and /supply pages Delivers - Repo set up (Day 1 morning) - ABI exported for the frontend (Day 1) - Local demo chain running (Day 1) - Admin and supply pages (Day 2) - Contract live and verified on Sepolia (Day 3 morning) Skills for your CV Solidity, OpenZeppelin access control, Foundry scripts, cast, deploying and verifying contracts	Name: _____ Owns - frontend/ (everything except /admin and /supply) - Vercel hosting Delivers - Wallet setup + verify page on the local chain (Day 1) - Manufacturer page with QR sheet, pharmacy page (Day 2) - Verify page polished for phones (Day 2–3) - Live site on Vercel using Sepolia (Day 3) Skills for your CV React / Next.js, wagmi and viem, wallet UX, reading and writing contracts, deployment	Name: _____ Owns - contracts/test, docs/security-report.md .github/workflows (CI) - Demo video, slides and submission Delivers - All tests running and understood (Day 1) 100% branch coverage (Day 2) - Slither report + security report (Day 2) - Demo script + slides draft (Day 3) - Final video, rehearsal, submission (Day 4) Skills for your CV Smart-contract security, fuzz and invariant testing, static analysis, presenting technical work

2.1 Day-by-day plan

Day	Member 1 · Blockchain	Member 2 · Frontend	Member 3 · Security & presentation
Day 1 Sat 26 Sep	Set up the repo (Section 5). Add the contract, export the ABI, run the local demo script.	Install tools. Add wagmi config and providers. Build the /verify page against the local chain.	Install Foundry and Slither. Add the test files, run them, study the threat model (Section 9).
	Everyone requests Sepolia test ETH. Evening check: /verify shows GENUINE, ALREADY SOLD and NOT FOUND locally.		
Day 2 Sun 27 Sep	Build /admin (register participants, approve, recall) and the transfer form on /supply.	Build /manufacturer (register a batch, QR sheet) and /pharmacy (sell a pack, friendly errors).	Raise branch coverage to 100%. Run Slither, write the security report, add CI.
	Evening check: the whole flow works in the browser with MetaMask on the local chain.		
Day 3 Mon 28 Sep	Deploy and verify on Sepolia. Register demo participants, fund the demo accounts.	Deploy to Vercel with the Sepolia address. Polish the verify page on a phone.	Write the demo script, print QR props, draft the slides.
	Evening check: a full dry run on Sepolia, scanning with a real phone.		
Day 4 Tue 29 Sep	Fix bugs. Put addresses and links in the README.	Fix UI bugs. Help record the demo.	Record the demo video, finish the slides, name the file, submit on ELMS .
	Two timed rehearsals (3:00 or less). Submit by the evening.		

Day	Member 1 · Blockchain	Member 2 · Frontend	Member 3 · Security & presentation
Wed 30 Sep	Buffer only. No new features.		
	Confirm the real deadline with the coordinator.		

2.2 Handoffs between members

#	When	From → to	What is handed over
H1	Day 1, midday	Member 1 → 2, 3	Contract and frontend/lib/registryAbi.ts pushed to main
H2	Day 1, evening	Member 1 → 2	local-demo.sh works, giving three test links for the verify page
H3	Day 3, morning	Member 1 → 2, 3	Sepolia contract address + Etherscan link (in the README)
H4	Day 3, afternoon	Member 2 → 3	Live Vercel URL, used in the QR codes and the demo video

2.3 How we use Git as a team

- **main must always work.** Each member works on their own branch: `m1-contracts`, `m2-frontend`, `m3-tests`.
- Start each day with `git pull origin main`, then merge main into your branch.
- Merge through a Pull Request on GitHub. Another member skims and approves it. If nobody is available within about 2 hours, merge it yourself, but only if the tests pass.
- Make small commits with clear messages, e.g. "Add dispense form to pharmacy page".
- **Never commit** .env files, private keys, recovery phrases or real serial lists.

2.4 Definition of done

- ☐ `forge test` passes and `forge fmt --check` passes
- ☐ Slither shows no medium or high findings, and the accepted low findings are documented
- ☐ Contract deployed and verified on Sepolia, with the Etherscan link in the README
- ☐ Web app live on Vercel. The verify page works on a phone with no wallet
- ☐ Full demo flow completed on Sepolia at least twice
- ☐ Demo video (about 70 seconds) recorded, slides done, rehearsed to 3:00 or less
- ☐ File named **GP_XX_Counterfeit_Medicine_Checker.ppt** (our group number) submitted on ELMS
- ☐ README explains what the project is and how to run it (this is our portfolio piece)

3 Design decisions (everyone should understand these)

3.1 Who does what

Role in contract	Real-world party	Can do	Needs a wallet?
REGULATOR_ROLE	NMRA	Register/revoke participants; approve and recall batches	Yes
MANUFACTURER_ROLE	Licensed drug maker	Register batches; ship them; recall own batch	Yes
DISTRIBUTOR_ROLE	Wholesaler	Ship batches on to pharmacies	Yes
PHARMACY_ROLE	Licensed pharmacy	Mark packs as sold (dispense)	Yes
(none)	Patient / public	Verify any pack by scanning its QR code	No , reading is free

Demo names such as “CeyPharma Ltd”, “Island Distributors” and “Pharmacy A, Galle” are made up. We don’t use real company names.

3.2 Lifecycle of a batch and a pack

- **Batch status:** Pending → Approved (by the NMRA) → Recalled (by the NMRA or its own manufacturer).
- **Custody:** Manufacturer → Distributor → Pharmacy. Each handover is stored with a timestamp and returned by `getTrail(batchId)`.
- **Pack:** Registered → Sold. A pack can be sold **only once**, and only by the pharmacy that currently holds its batch.

Verdict	When	What the patient sees
NotFound	The serial was never registered	NOT FOUND: likely counterfeit
NotApproved	Registered, but the NMRA hasn’t approved it yet	Not yet approved by NMRA
Genuine	Approved, in date, not yet sold	GENUINE + medicine, batch and expiry
Dispensed	Already sold at a pharmacy	ALREADY SOLD at <i>Pharmacy A</i> on <i>date</i> . Genuine only if that’s where and when you bought it
Recalled	The NMRA or the manufacturer recalled the batch	RECALLED: do not use
Expired	Past the expiry date	EXPIRED

If several apply, the contract picks the most serious: Recalled, then NotApproved, then Dispensed, then Expired, then Genuine.

3.3 How serials and QR codes keep packs safe

1. Each pack gets a **random 128-bit serial**, e.g. `0x8f3a1c2b4d5e6f708192a3b4c5d6e7f8`, generated in the manufacturer’s browser.
2. At registration, **only the hash** (keccak256) of each serial goes on-chain. Nobody can read serials off the blockchain and print valid fake packs.
3. The QR code holds a link: `https://<our-site>/verify?s=<serial>`. Any phone camera opens it, with no app and no wallet.
4. To sell a pack, the pharmacy must submit the **raw serial** (proving it has the physical pack) **and** be the current holder of that batch.
5. Once sold, anyone who scans a copy of that code sees where and when it was sold.

3.4 Contract functions at a glance

Function	Called by	What it does	Rejects when
registerParticipant	NMRA	Gives an address a role and a display name	Caller isn't NMRA; role isn't manufacturer, distributor or pharmacy
revokeParticipant	NMRA	Removes a role, which freezes that party's stock	Caller isn't NMRA
registerBatch	Manufacturer	Creates a batch of 1–200 packs (serial hashes)	Not a manufacturer; 0 or more than 200 packs; expiry in the past; serial already used
approveBatch	NMRA	Lets the batch move and be sold	Not NMRA; batch isn't pending
recallBatch	NMRA or its manufacturer	Stops movement and sale	Anyone else; already recalled
transferBatch	Current holder	Hands the batch to a distributor or pharmacy	Not approved; not the holder; role revoked; expired; recipient not registered
dispense	Pharmacy holding the batch	Marks one pack as sold	Unknown serial; already sold; not approved or recalled; expired; not the holder
verify	Anyone (free)	Returns the verdict, batch details and who sold it when	Never reverts
getBatch, getTrail, orgName, hashSerial	Anyone (free)	Details for the web app	(read-only)

3.5 Known limitations (be ready for these in Q&A)

- **First-scan race:** if a counterfeiter copies a code from a genuine pack on a shelf and sells the fake first, the *genuine* pack later shows “already sold”. The fraud is still detected, and we know which pharmacy sold it. In production, the QR goes inside the box or under a scratch panel.
- **Batch-level custody:** in the demo, a whole batch goes to one pharmacy. Real systems split batches into shipments (a stretch goal).
- **200 packs per registration** keeps each transaction within gas limits. A Merkle-tree version could register any batch size with a single hash (a stretch goal).
- **Fees:** supply-chain parties pay small transaction fees; patients pay nothing. A real deployment would use a low-fee Ethereum layer-2 network or a permissioned chain.

4 Tools and accounts (everyone, first two hours)

4.1 Install

Tool	Who	Why	Check it works
Git for Windows (includes Git Bash)	All	Version control. Run every command in this guide in Git Bash , not PowerShell.	<code>git --version</code>
Node.js LTS (v22 or newer)	All	The web app and the OpenZeppelin package	<code>node --version</code>
VS Code + "Solidity" extension (Nomic Foundation)	All	Editing with Solidity highlighting and errors	
Foundry (forge, cast, anvil)	All	Compile, test and deploy; anvil is the local blockchain Member 2 also needs	<code>forge --version</code>
MetaMask browser extension	All	Signs transactions as the NMRA, manufacturer, etc.	
Python 3 + Slither	Member 3	Static security analysis	<code>slither --version</code>

Git Bash	commands
<pre># Foundry (Git Bash). Its installer doesn't support PowerShell or CMD. curl -L https://foundry.paradigm.xyz bash # close and reopen Git Bash, then: foundryup forge --version # Windows: allow long file paths in Git (dependency installs failed without this in our test) git config --global core.longpaths true # Member 3 only python -m pip install slither-analyzer slither --version</pre>	

If foundryup doesn't work on your laptop

Download **foundry_v1.8.3_win32_amd64.zip** from the Releases page of github.com/foundry-rs/foundry, unzip it to **C:\foundry**, and add that folder to your Windows PATH. This is exactly how the tools were run while testing this guide.

4.2 Accounts

Account	Who	What to do
MetaMask demo wallet	Member 1 (demo laptop)	Create a new wallet used only for this project. Add 4 accounts and rename them NMRA , CeyPharma , Distributor and Pharmacy A . Turn on "Show test networks" and select Sepolia .
Alchemy (alchemy.com)	Member 1	Create a free app on Ethereum Sepolia and copy its HTTPS URL. This is our <code>SEPOLIA_RPC_URL</code> and <code>NEXT_PUBLIC_RPC_URL</code> .
Etherscan (etherscan.io)	Member 1	Sign up, go to API Keys and create one (<code>ETHERSCAN_API_KEY</code>). One key also works for Sepolia.
Vercel (vercel.com)	Member 2	Sign in with GitHub. We host the web app here for free.

Account	Who	What to do
Sepolia test ETH	Everyone, today	Use a Sepolia faucet (e.g. the Google Cloud Web3 faucet) and send the test ETH to the NMRA address. Faucets limit each person per day, so all three should request it. Aim for about 0.1 test ETH in total. Deploying uses about 3.1 million gas.

Wallet safety rules (they also count as “security awareness” marks)

- Never put real money in the demo wallet, and never use your personal wallet for this project.
- Never paste a recovery phrase or private key into chat, a commit or a screenshot.
- On Sepolia we deploy with an **encrypted keystore** (`cast wallet import`), so the private key is never stored in a plain .env file.
- Anvil’s built-in test keys are public. Use them only on your local chain.

5 Repository setup (Member 1, once, about 30 minutes)

These commands were run and tested. Use a **short folder path** (for example D:\Projects). Very long paths caused "Filename too long" errors in testing.

5.1 Create both projects inside the repo

```
Git Bash commands

# 1. Clone into a short path
cd /d/Projects
git clone https://github.com/Raviyaa27/counterfeit-medicine-checker.git
cd counterfeit-medicine-checker

# 2. Smart-contract project (Foundry) inside the SAME repo
forge init contracts --use-parent-git
cd contracts
rm src/Counter.sol test/Counter.t.sol script/Counter.s.sol
rm -rf .github # GitHub only runs workflows from the repo root
npm init -y
npm install --save-exact @openzeppelin/contracts@5.6.1
echo "node_modules/" >> .gitignore
cd ..

# 3. Web app (Next.js 16)
npx create-next-app@latest frontend --ts --tailwind --eslint --app --use-npm --disable-git --yes
cd frontend
npm install wagmi viem @tanstack/react-query qrcode.react
cd ..
```

Why these particular flags?

- **--use-parent-git**: without it, forge creates a second Git repo inside contracts/, which breaks committing and pushing.
- **OpenZeppelin from npm** instead of **forge install**: the forge version pulls several nested Git submodules and failed on Windows in our test. The npm package contains only the contracts we need.
- **--disable-git**: we already have a Git repo, so we don't want another one.

5.2 Add the files from the appendices

File	From	Notes
contracts/foundry.toml	Appendix A	Replaces the generated one (compiler, remappings, fuzz settings, Sepolia)
contracts/src/MedicineRegistry.sol	Appendix A	The smart contract
contracts/script/Deploy.s.sol	Appendix A	Deploy script
contracts/script/local-demo.sh	Appendix D	One command to deploy and add demo data locally
contracts/slither.config.json	Appendix A	Tells Slither to skip libraries and tests
contracts/.env.example, frontend/.env.example	Appendix A / C	Templates. Real .env files are never committed
contracts/test/*.t.sol	Appendix B	Member 3 can add these later on their own branch

```
Git Bash commands

cd contracts
forge build # should say "Compiler run successful"
cd ..
```

Git Bash	commands
<pre>git add . git commit -m "Set up Foundry and Next.js projects" git push origin main</pre>	

5.3 Teammates: clone and install

Git Bash	commands
<pre>cd /d/Projects git clone --recursive https://github.com/Raviyaa27/counterfeit-medicine-checker.git cd counterfeit-medicine-checker/contracts && npm install && forge build cd ../frontend && npm install # If you cloned without --recursive: git submodule update --init --recursive</pre>	

5.4 Final folder structure

repository layout	tree
<pre>counterfeit-medicine-checker/ ├── contracts/ Foundry project (Members 1 and 3) │ ├── src/MedicineRegistry.sol │ ├── test/ unit, attack, fuzz and invariant tests │ ├── script/ Deploy.s.sol, local-demo.sh │ ├── foundry.toml slither.config.json .env.example │ └── lib/forge-std node_modules/ (dependencies, not edited) ├── frontend/ Next.js app (Member 2; Member 1 adds /admin, /supply) │ ├── app/ verify/, manufacturer/, pharmacy/, admin/, supply/ │ ├── components/ VerifyResult, NewBatchForm, DispenseForm, ... │ └── lib/ config.ts, registryAbi.ts, roles.ts, tx.ts ├── docs/ this guide, security report, slides, diagrams └── .github/workflows/ci.yml automatic tests on every push (Member 3)</pre>	

6 Member 1 track: blockchain lead

M1.1 Build the contract and read it carefully

After the setup in Section 5, run the command below. Then read the contract in Appendix A from top to bottom. You'll be asked about it in Q&A: roles, the checks in `dispense`, and why only hashes are stored.

Git Bash	commands
<pre>cd contracts forge build forge inspect MedicineRegistry abi --json head -20 # peek at the interface</pre>	

M1.2 Export the ABI for the web app (repeat after every contract change)

The ABI describes the contract's functions to the web app. We export it as a TypeScript file with **as const** so wagmi can check argument types for us.

Git Bash	commands
<pre>cd contracts echo "export const registryAbi = \$(forge inspect MedicineRegistry abi --json) as const;" \ > ../frontend/lib/registryAbi.ts</pre>	

Handoff H1: push this to main

Tell Members 2 and 3 when the contract and `registryAbi.ts` are on main. Member 2 builds on top of the ABI, and Member 3 writes tests against the contract.

M1.3 Run everything on a local blockchain

Anvil is a private blockchain on your laptop with 10 pre-funded test accounts. Transactions are instant and free. The script deploys the contract, registers the three demo companies, creates a batch of two packs, moves it to the pharmacy and sells one pack.

Git Bash	commands
<pre># Terminal 1 (leave it running) anvil # Terminal 2 cd contracts bash script/local-demo.sh # Registry: 0x5FbDB2315678afecb367f032d93F642f64180aa3 # Genuine: http://localhost:3000/verify?s=0x8f3a1c2b4d5e6f708192a3b4c5d6e7f8 # Sold: http://localhost:3000/verify?s=0x11112222333344445555666677778888 # Fake: http://localhost:3000/verify?s=0xdeadbeefdeadbeefdeadbeefdeadbeef</pre>	

Handoff H2

Send these three links to Member 2. When their verify page shows GENUINE, ALREADY SOLD and NOT FOUND, Day 1 is a success. Restarting Anvil wipes the chain, so just run the script again.

M1.4 Build the NMRA admin page (/admin)

Three small forms. Copy the **simulate** → **write** → **wait** pattern from `DispenseForm.tsx` (Appendix C) and change the function name and arguments.

Form	Inputs	Contract call
Register participant	Address, role (dropdown), organisation name	<code>registerParticipant(address, ROLES.Manufacturer, name)</code>
Approve batch	Batch ID	<code>approveBatch(BigInt(id))</code>
Recall batch	Batch ID, reason	<code>recallBatch(BigInt(id), reason)</code>

frontend/lib/roles.ts	TypeScript
<pre>import { keccak256, toHex } from "viem"; // Same values as the role constants in MedicineRegistry.sol export const ROLES = { Manufacturer: keccak256(toHex("MANUFACTURER_ROLE")), Distributor: keccak256(toHex("DISTRIBUTOR_ROLE")), Pharmacy: keccak256(toHex("PHARMACY_ROLE")), } as const;</pre>	

Tip: read `batchCount` and loop over `getBatch(id)` to show a list of pending batches with an Approve button next to each. It looks great in the demo.

M1.5 Build the transfer form (/supply)

A batch ID plus a “ship to” dropdown of known participants, calling `transferBatch(id, to)`. Used by the manufacturer (to the distributor) and the distributor (to the pharmacy). Show the custody history using `getTrail(id)`.

M1.6 Deploy to Sepolia and verify on Etherscan (Day 3 morning)

Git Bash	commands
<pre>cd contracts # One-time: store the NMRA account's key ENCRYPTED (you'll be asked for the key and a password). # MetaMask: select NMRA > Account details > Show private key (demo wallet only!) cast wallet import nmra --interactive cp .env.example .env # fill in SEPOLIA_RPC_URL, ETHERSCAN_API_KEY, REGULATOR_ADDRESS source .env forge script script/Deploy.s.sol --rpc-url sepolia --account nmra \ --sender \$REGULATOR_ADDRESS --broadcast --verify</pre>	

Open sepolia.etherscan.io, search the printed address, and check that the **Contract** tab shows a green tick (“Source Code Verified”).

M1.7 Register the demo participants on Sepolia and fund them

Git Bash	commands
<pre>REG=<deployed contract address> cast send \$REG "registerParticipant(address,bytes32,string)" <CeyPharma address> \ \$(cast keccak MANUFACTURER_ROLE) "CeyPharma Ltd" --account nmra --rpc-url \$SEPOLIA_RPC_URL # Repeat with DISTRIBUTOR_ROLE "Island Distributors" and PHARMACY_ROLE "Pharmacy A, Galle" # (or use your /admin page). Then, in MetaMask, send about 0.01 test ETH from NMRA to each # of the other three demo accounts so they can pay gas.</pre>	

Handoff H3

Put the contract address and its Etherscan link at the top of the README, and send them to Members 2 and 3.

7 Member 2 track: frontend lead

M2.1 Connect the app to the blockchain

Add these files from Appendix C: **lib/config.ts**, **lib/tx.ts**, **app/providers.tsx**, and the updated **app/layout.tsx** (it wraps every page in the providers). Then create your local settings:

```
Git Bash commands  
  
cd frontend  
cp .env.example .env.local      # points at the local Anvil chain  
npm run dev                     # open http://localhost:3000
```

M2.2 Set up MetaMask for local testing

1. MetaMask → Networks → **Add a network manually**: name **Anvil**, RPC URL <http://127.0.0.1:8545>, chain ID **31337**, currency **ETH**.
2. **Import account** four times using Anvil's test keys (printed when anvil starts, and in local-demo.sh). Rename them NMRA, Manufacturer, Distributor, Pharmacy.
3. After restarting Anvil: MetaMask → Settings → Advanced → **Clear activity tab data**. This fixes "nonce too high" errors.

M2.3 The verify page: the most important screen

Add **app/verify/page.tsx**, **components/VerifyResult.tsx** and **components/OrgName.tsx** (Appendix C). The page reads the contract directly, so it needs no wallet. That's what lets any patient use it. Test it with the three links from Member 1's local-demo.sh.

Expected result (tested)

Genuine link → green **GENUINE** with medicine, batch and expiry. Sold link → amber **ALREADY SOLD** with "Sold by Pharmacy A, Galle on ...". Fake link → red **NOT FOUND: likely counterfeit**.

Then make it demo-ready:

- A big coloured result card and large text. It will be viewed on a phone, so test at phone width.
- Show the custody trail: call `getTrail(batchId)` and list each handover with `<OrgName address={...} />` and the date.
- Add a line such as "Checked on the Ethereum Sepolia blockchain" with a link to the contract on Etherscan. It builds trust.

M2.4 Manufacturer page (/manufacturer)

Start from **NewBatchForm.tsx** (Appendix C), which generates random serials, registers their hashes, and then shows one QR code per pack. Extend it:

- Inputs for drug name, batch number, expiry date and number of packs (1–200).
- A **Download serials (CSV)** button, and a print-friendly QR sheet (a CSS `@media print` rule that hides everything except the QR grid).
- Show the new batch ID. The NMRA approves it next.

Serials are secrets

Anyone with the serial list can print codes that pass as genuine. Never commit the CSV or post it in chat. For the demo, only print what you need.

M2.5 Pharmacy page (/pharmacy)

Start from `DispenseForm.tsx` and the error translator `friendlyError` in `lib/tx.ts`. Contract errors such as `AlreadyDispensed` become messages like “Clone alert: this pack was already sold”.

Nice-to-have: a camera scanner. Run `npm install @yudiel/react-qr-scanner`, then read the serial from the scanned link with `new URL(text).searchParams.get("s")`.

M2.6 Landing page (/)

Project name, a one-sentence pitch, a “How to check a medicine” box and links to the four pages. This is the first thing examiners see.

M2.7 Deploy to Vercel (Day 3, after H3)

1. Vercel → **Add New...** → **Project** → import the GitHub repo.
2. Set **Root Directory** to **frontend**.
3. Environment variables: `NEXT_PUBLIC_CHAIN_ID=11155111`, `NEXT_PUBLIC_REGISTRY_ADDRESS=<Sepolia address>`, `NEXT_PUBLIC_RPC_URL=<Alchemy URL>`.
4. Deploy. If you change an environment variable later, click **Redeploy**, because `NEXT_PUBLIC_` values are baked in at build time.
5. Open the live verify link on a phone, using mobile data rather than the university Wi-Fi.

Note: `NEXT_PUBLIC_` values are visible to anyone who visits the site. If your RPC provider supports it, restrict the key to your Vercel domain.

Handoff H4

Send the live URL to Member 3 for the QR props and the demo recording.

8 Member 3 track: security, testing and presentation lead

M3.1 Add and run the tests

Copy both test files from Appendix B into **contracts/test/**. There are four kinds of test:

Type	Example	What it proves
Unit	test_GenuinePackVerifies	Normal behaviour works
Attack	test_Attack_CannotSellSamePackTwice	A specific attack is blocked (9 of these)
Fuzz	testFuzz_RandomSerialIsNotFound	Holds for 1,000 random inputs each
Invariant	invariant_NoPackIsSoldTwice	Holds after 12,800 random calls in random order

Git Bash	commands
<pre>cd contracts forge test # expected: 20 tests passed, 0 failed forge test --match-test Attack -vv # the 9 attack tests: screenshot this for the slides</pre>	

M3.2 Raise branch coverage from 67% to 100%

Git Bash	commands
<pre>forge coverage --report summary --no-match-coverage "(test script)" # today: about 98% of lines but only 67% of branches</pre>	

Add a test for each of these cases. Together they were checked to reach **100% of lines, branches and functions**:

Case to test	Expected error
Approve the same batch twice	InvalidStatus(Approved)
Approve a batch ID that doesn't exist (e.g. 99)	UnknownBatch(99)
Stranger tries to recall; manufacturer recalls own batch (allowed); recall again	NotAuthorized, then InvalidStatus(Recalled)
Transfer before approval	InvalidStatus(Pending)
Transfer by someone who isn't the holder	NotHolder(holder)
Transfer to yourself	InvalidRecipient(you)
Pharmacy tries to transfer onward	NotAuthorized
Transfer or dispense after expiry (vm.warp)	BatchExpired(expiry)
Dispense an unknown serial; dispense from a pending batch	UnknownSerial, InvalidStatus(Pending)
Register a participant with REGULATOR_ROLE	InvalidRole(role)

M3.3 Static analysis with Slither and the security report

Git Bash	commands
<pre>cd contracts slither . # uses slither.config.json # expected: "102 detectors, 4 result(s) found", all 'timestamp' (low severity)</pre>	

Write **docs/security-report.md** with: scope (one contract), tools (Foundry, Slither, forge lint), a findings table (detector, severity, location, decision), the threat model from Section 9, and your test evidence (test count, coverage,

invariant runs). The 4 timestamp findings are **accepted**: validators can shift block time by seconds, but expiry dates are measured in days.

M3.4 Gas report (one number for the slides)

```
Git Bash commands
forge test --gas-report
```

Action	Gas (measured)
Deploy the contract	about 3.1 million
Register a participant	about 76,000
Register a batch	about 188,000 for 1 pack, plus about 23,000 per extra pack
Approve a batch / transfer a batch	about 31,000 / about 104,000
Sell a pack (dispense)	about 60,000
Verify a pack (patient)	free : it's a read, not a transaction

M3.5 Continuous integration (optional, good for your CV)

Add `.github/workflows/ci.yml` from Appendix D. Every push then runs the formatter check, build, all tests and Slither on GitHub. A green tick in the README looks professional.

M3.6 Demo, slides and submission

You own Section 10: print the QR props, record the demo video, build the slides, run rehearsals, and submit the correctly named file on ELMS.

9 Security: threat model

This table is the backbone of the security slide. Each defence is enforced by the contract and proved by a named test.

Threat	How the contract stops it	Test that proves it
Unlicensed factory registers a fake batch	Only MANUFACTURER_ROLE can register	test_Attack_UnregisteredFactoryCannotRegisterBatch, testFuzz_OnlyManufacturersCanRegister
Made-up QR codes	Unknown serials give NotFound	test_Attack_FakeCodeIsNotFound, testFuzz_RandomSerialIsNotFound
Photocopied genuine QR code	After a sale, every scan shows where and when it was sold; a second sale is rejected	test_Attack_ClonedQrCodeIsFlaggedAfterSale, test_Attack_CannotSellSamePackTwice, invariant_NoPackIsSoldTwice
Rogue pharmacy sells stock it never received	Needs the raw serial and must hold the batch	test_Attack_RoguePharmacyCannotSellOthersStock
Manufacturer re-uses another pack's serial	Duplicate serial hashes are rejected	test_Attack_CannotHijackExistingSerial
Recalled medicine keeps being sold	Recalled batches can't move or be sold; verify shows RECALLED	test_Attack_RecalledBatchCannotBeSold
Struck-off distributor keeps shipping	Revoking the role blocks transfers	test_Attack_RevokedDistributorCannotShip
Stock diverted to an unlicensed seller	Recipient must be a registered distributor or pharmacy	test_Attack_CannotShipToUnregisteredAddress
Huge batch to exhaust gas (denial of service)	Maximum of 200 packs per call	test_RevertWhen_BatchIsEmptyOrTooLarge
Serial list read from the blockchain	Only hashes are stored; guessing a 128-bit serial is infeasible	Design (Section 3.3)
Stolen deployer key	Encrypted keystore, never in .env. In production: a multi-signature NMRA wallet	Process

Accepted risks (from Slither and forge lint)

- **block.timestamp comparisons** (4 low findings): validators can shift time by a few seconds; expiry is measured in days.
- **Revert inside a loop** in registerBatch: intentional. A batch with a duplicate serial is rejected completely (all-or-nothing).
- **No reentrancy risk**: the contract never sends ETH or calls other contracts.

Privacy

No patient data is ever stored on-chain, only medicine, batch and organisation data plus serial hashes. This matters under Sri Lanka's Personal Data Protection Act (No. 9 of 2022). Say it in the presentation.

10 Demo and presentation

10.1 Props

- An empty medicine box with a **genuine** QR code that stays unsold.
- A second box that you **sell during the demo**, plus a **photocopy** of its QR code.
- A box with a **fake** code (a random serial that was never registered).

10.2 Demo video storyboard (about 70 seconds, on Sepolia)

Time	On screen	Say
0:00	NMRA /admin: a pending batch → Approve	"The NMRA approves a batch. Only licensed manufacturers can register one."
0:10	/supply: manufacturer → distributor → pharmacy; custody trail	"Every handover is recorded on the blockchain."
0:20	Phone scans the genuine box → green GENUINE	"A patient scans the pack. No app, no wallet, and it's free."
0:32	/pharmacy: scan → Mark as sold	"At the counter, the pharmacy marks it sold."
0:42	Phone scans the photocopy → ALREADY SOLD at Pharmacy A	"A counterfeiter copied that code. The copy is caught instantly."
0:55	Fake box → red NOT FOUND	"A made-up code is rejected."
1:03	Etherscan page with the transactions	"Every step is public and tamper-proof on Ethereum."

10.3 3-minute slide plan

Time	Slide	Content
0:00–0:25	Problem	WHO: 1 in 10 medical products in low- and middle-income countries; the 2023 Sri Lanka immunoglobulin scandal
0:25–0:45	Why blockchain	Many parties, no single trusted database; records can't be changed; anyone can verify
0:45–1:05	How it works	The diagram from Section 1.2
1:05–2:15	Demo	The 70-second video (embedded, with a backup MP4)
2:15–2:45	Security	Four rows of the threat table; "20 tests incl. 12,800-call invariant test; Slither: no medium/high issues"
2:45–3:00	Impact	QR code to the live site and GitHub; future work: layer-2 network, NMRA integration

File name and format

Name it **GP_XX_Counterfeit_Medicine_Checker.ppt** with our group number. The brief asks for .ppt, but the old .ppt format may not keep an embedded video. Ask the coordinator whether .pptx is accepted, and keep a backup MP4.

10.4 Likely examiner questions

Question	Short answer
Why not a normal database?	Several organisations that don't fully trust each other write to it. No single admin can edit or delete records, and anyone can verify without permission.

Question	Short answer
What stops someone copying a QR code?	A copy can't be sold twice. Once sold, every scan shows where and when. In production the code goes inside the box or under a scratch panel.
Who pays the fees? Patients?	Patients pay nothing (reads are free). Supply-chain parties pay small fees, which would be tiny on a layer-2 network.
Is personal data on the blockchain?	No. Only medicine and batch data plus hashes, in line with the PDPA (No. 9 of 2022).
What if the NMRA's key is stolen?	In production the regulator role would be a multi-signature wallet needing several officials, and roles can be revoked.
Why a public chain rather than Hyperledger?	Public verifiability with no accounts. A permissioned chain is a valid production choice, and the contract logic would be similar.
How does it scale to millions of packs?	Registration is capped at 200 packs per call. A Merkle-root version registers any batch size with one hash.
How did you test security?	20 automated tests incl. 9 attack scenarios, fuzzing, a 12,800-call invariant test, and Slither static analysis.

11 Troubleshooting

Problem	Fix
forge: command not found	Close and reopen Git Bash after foundryup, or use the zip method in Section 4.1.
Filename too long	<code>git config --global core.longpaths true</code> and keep the repo in a short path.
forge-std/Test.sol not found	The submodule is missing: <code>git submodule update --init --recursive</code> .
@openzeppelin/... not found	Run <code>npm install</code> inside contracts/.
MetaMask: “nonce too high” after restarting Anvil	Settings → Advanced → Clear activity tab data.
Web app still talks to the old contract	Update NEXT_PUBLIC_REGISTRY_ADDRESS and restart npm run dev (on Vercel: Redeploy).
“function not found” or wrong arguments in the app	The ABI is out of date. Re-run the export command in M1.2.
“Your wallet doesn’t have permission for this action”	The wrong MetaMask account is selected (e.g. the Pharmacy account trying to approve).
The wallet’s network doesn’t match the app	Switch MetaMask to Anvil (local) or Sepolia (live) to match NEXT_PUBLIC_CHAIN_ID.
Faucet asks for a mainnet balance	Try another faucet (e.g. Google Cloud Web3), or ask a teammate to send test ETH.
Deploy says “insufficient funds”	The NMRA account needs test ETH. Deploying uses about 3.1M gas.
Etherscan verification failed	Check ETHERSCAN_API_KEY, then retry with <code>forge verify-contract</code> (see <code>forge verify-contract --help</code>).

12 Stretch goals (only after the definition of done)

- Camera QR scanner on the pharmacy page.
- “Report suspicious pack” button for patients (gasless, via a small server that pays the fee).
- Merkle-root batches: register 100,000 packs with one hash.
- Split batches into shipments so one batch can go to several pharmacies.
- Sinhala and Tamil versions of the verify page.
- Multi-signature NMRA wallet (Safe) and deployment on a low-fee layer-2 network.

Appendix A Smart-contract files

Tested with Foundry 1.8.3, solc 0.8.30 and OpenZeppelin 5.6.1. Copy each file into the path shown in its header.

contracts/foundry.toml

TOML

```
[profile.default]
src = "src"
out = "out"
libs = ["lib", "node_modules"]
solc = "0.8.30"
remappings = ["@openzeppelin/contracts/=node_modules/@openzeppelin/contracts/"]

[fuzz]
runs = 1000

[invariant]
runs = 256
depth = 50

[_rpc_endpoints]
sepolia = "${SEPOLIA_RPC_URL}"

[etherscan]
sepolia = { key = "${ETHERSCAN_API_KEY}" }

[fmt]
line_length = 100
```

contracts/src/MedicineRegistry.sol

Solidity

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import {AccessControl} from "@openzeppelin/contracts/access/AccessControl.sol";

/// @title MedicineRegistry
/// @notice Tracks medicine batches from manufacturer to pharmacy. Anyone can verify a pack
///         using the secret serial printed in its QR code.
contract MedicineRegistry is AccessControl {
    // ----- roles
    bytes32 public constant REGULATOR_ROLE = keccak256("REGULATOR_ROLE"); // NMRA
    bytes32 public constant MANUFACTURER_ROLE = keccak256("MANUFACTURER_ROLE");
    bytes32 public constant DISTRIBUTOR_ROLE = keccak256("DISTRIBUTOR_ROLE");
    bytes32 public constant PHARMACY_ROLE = keccak256("PHARMACY_ROLE");

    /// @dev Caps the loop in registerBatch so a call can never run out of gas.
    uint256 public constant MAX_UNITS_PER_BATCH = 200;

    // ----- types
    enum BatchStatus {
        None,
        Pending,
        Approved,
        Recalled
    }

    enum Verdict {
        NotFound, // serial was never registered: likely counterfeit
        NotApproved, // batch not yet approved by the regulator
        Genuine, // approved, in date, not yet sold
        Dispensed, // already sold: genuine only if the buyer bought it there
        Recalled, // do not use
        Expired
    }

    struct Batch {
        string drugName;
        string batchNumber;
        address manufacturer;
    }
```

```

        address holder;
        uint64 expiryDate;
        uint32 unitCount;
        uint32 dispensedCount;
        BatchStatus status;
    }

    struct Unit {
        uint256 batchId; // 0 means "not registered"
        address dispensedBy;
        uint64 dispensedAt;
    }

    struct Handover {
        address from;
        address to;
        uint64 at;
    }

    // ----- storage
    uint256 public batchCount;
    mapping(uint256 => Batch) private _batches;
    mapping(uint256 => Handover[]) private _trail;
    mapping(bytes32 => Unit) private _units; // key: keccak256(serial)
    mapping(address => string) public orgName;

    // ----- events
    event ParticipantRegistered(address indexed account, bytes32 indexed role, string name);
    event ParticipantRevoked(address indexed account, bytes32 indexed role);
    event BatchRegistered(
        uint256 indexed batchId, address indexed manufacturer, string batchNumber, uint256 units
    );
    event BatchApproved(uint256 indexed batchId);
    event BatchRecalled(uint256 indexed batchId, address indexed by, string reason);
    event BatchTransferred(uint256 indexed batchId, address indexed from, address indexed to);
    event UnitDispensed(
        uint256 indexed batchId, bytes32 indexed serialHash, address indexed pharmacy
    );

    // ----- errors
    error InvalidRole(bytes32 role);
    error InvalidUnitCount(uint256 count);
    error InvalidExpiry();
    error DuplicateSerial(bytes32 serialHash);
    error UnknownBatch(uint256 batchId);
    error InvalidStatus(BatchStatus status);
    error NotAuthorized();
    error NotHolder(address holder);
    error BatchExpired(uint64 expiryDate);
    error InvalidRecipient(address to);
    error UnknownSerial();
    error AlreadyDispensed(address pharmacy, uint64 at);

    constructor(address regulator) {
        _grantRole(DEFAULT_ADMIN_ROLE, regulator);
        _grantRole(REGULATOR_ROLE, regulator);
    }

    // ----- regulator (NMRA)

    function registerParticipant(address account, bytes32 role, string calldata name)
        external
        onlyRole(REGULATOR_ROLE)
    {
        if (role != MANUFACTURER_ROLE && role != DISTRIBUTOR_ROLE && role != PHARMACY_ROLE) {
            revert InvalidRole(role);
        }
        _grantRole(role, account);
        orgName[account] = name;
        emit ParticipantRegistered(account, role, name);
    }

```

```

    }

    function revokeParticipant(address account, bytes32 role) external onlyRole(REGULATOR_ROLE) {
        _revokeRole(role, account);
        emit ParticipantRevoked(account, role);
    }

    function approveBatch(uint256 batchId) external onlyRole(REGULATOR_ROLE) {
        Batch storage b = _existingBatch(batchId);
        if (b.status != BatchStatus.Pending) revert InvalidStatus(b.status);
        b.status = BatchStatus.Approved;
        emit BatchApproved(batchId);
    }

    /// @notice The regulator or the batch's own manufacturer can recall it.
    function recallBatch(uint256 batchId, string calldata reason) external {
        Batch storage b = _existingBatch(batchId);
        if (!hasRole(REGULATOR_ROLE, msg.sender) && msg.sender != b.manufacturer) {
            revert NotAuthorized();
        }
        if (b.status == BatchStatus.Recalled) revert InvalidStatus(b.status);
        b.status = BatchStatus.Recalled;
        emit BatchRecalled(batchId, msg.sender, reason);
    }

    // ----- manufacturer

    /// @param serialHashes keccak256 of each pack's secret serial.
    /// Raw serials never go on-chain at registration.
    function registerBatch(
        string calldata drugName,
        string calldata batchNumber,
        uint64 expiryDate,
        bytes32[] calldata serialHashes
    ) external onlyRole(MANUFACTURER_ROLE) returns (uint256 batchId) {
        uint256 n = serialHashes.length;
        if (n == 0 || n > MAX_UNITS_PER_BATCH) revert InvalidUnitCount(n);
        if (expiryDate <= block.timestamp) revert InvalidExpiry();

        batchId = ++batchCount;
        _batches[batchId] = Batch({
            drugName: drugName,
            batchNumber: batchNumber,
            manufacturer: msg.sender,
            holder: msg.sender,
            expiryDate: expiryDate,
            // forge-lint: disable-next-line(unsafe-typecast) safe: n <= 200
            unitCount: uint32(n),
            dispensedCount: 0,
            status: BatchStatus.Pending
        });

        for (uint256 i; i < n; ++i) {
            bytes32 h = serialHashes[i];
            if (_units[h].batchId != 0) revert DuplicateSerial(h);
            _units[h].batchId = batchId;
        }
        emit BatchRegistered(batchId, msg.sender, batchNumber, n);
    }

    // ----- supply chain

    /// @notice Hand a batch to the next party. Only manufacturers and distributors can ship.
    function transferBatch(uint256 batchId, address to) external {
        Batch storage b = _existingBatch(batchId);
        if (b.status != BatchStatus.Approved) revert InvalidStatus(b.status);
        if (b.holder != msg.sender) revert NotHolder(b.holder);
        if (!hasRole(MANUFACTURER_ROLE, msg.sender) && !hasRole(DISTRIBUTOR_ROLE, msg.sender)) {
            revert NotAuthorized();
        }
    }

```

```

    if (block.timestamp >= b.expiryDate) revert BatchExpired(b.expiryDate);
    if (to == msg.sender || (!hasRole(DISTRIBUTOR_ROLE, to) && !hasRole(PHARMACY_ROLE, to))) {
        revert InvalidRecipient(to);
    }

    b.holder = to;
    // forge-lint: disable-next-line(unsafe-typecast) safe: fits in uint64
    _trail[batchId].push(Handover({from: msg.sender, to: to, at: uint64(block.timestamp)}));
    emit BatchTransferred(batchId, msg.sender, to);
}

/// @notice Pharmacy marks a pack as sold. Needs the raw serial, so the pharmacy must
///         physically hold the pack, and must be the current holder of its batch.
function dispense(bytes16 serial) external onlyRole(PHARMACY_ROLE) {
    bytes32 h = hashSerial(serial);
    Unit storage u = _units[h];
    if (u.batchId == 0) revert UnknownSerial();
    if (u.dispensedAt != 0) revert AlreadyDispensed(u.dispensedBy, u.dispensedAt);

    Batch storage b = _batches[u.batchId];
    if (b.status != BatchStatus.Approved) revert InvalidStatus(b.status);
    if (block.timestamp >= b.expiryDate) revert BatchExpired(b.expiryDate);
    if (b.holder != msg.sender) revert NotHolder(b.holder);

    u.dispensedBy = msg.sender;
    // forge-lint: disable-next-line(unsafe-typecast) safe: fits in uint64
    u.dispensedAt = uint64(block.timestamp);
    b.dispensedCount += 1;
    emit UnitDispensed(u.batchId, h, msg.sender);
}

// ----- public reads

/// @notice What a patient sees after scanning a QR code. Free to call, no wallet needed.
function verify(bytes32 serialHash)
    external
    view
    returns (
        Verdict verdict,
        uint256 batchId,
        Batch memory batch,
        address dispensedBy,
        uint64 dispensedAt
    )
{
    Unit storage u = _units[serialHash];
    batchId = u.batchId;
    if (batchId == 0) return (Verdict.NotFound, 0, batch, address(0), 0);

    batch = _batches[batchId];
    dispensedBy = u.dispensedBy;
    dispensedAt = u.dispensedAt;

    if (batch.status == BatchStatus.Recalled) verdict = Verdict.Recalled;
    else if (batch.status == BatchStatus.Pending) verdict = Verdict.NotApproved;
    else if (dispensedAt != 0) verdict = Verdict.Dispensed;
    else if (block.timestamp >= batch.expiryDate) verdict = Verdict.Expired;
    else verdict = Verdict.Genuine;
}

function getBatch(uint256 batchId) external view returns (Batch memory) {
    return _batches[batchId];
}

function getTrail(uint256 batchId) external view returns (Handover[] memory) {
    return _trail[batchId];
}

function hashSerial(bytes16 serial) public pure returns (bytes32) {
    return keccak256(abi.encodePacked(serial));
}

```

contracts/src/MedicineRegistry.sol	Solidity
<pre> } // ----- internal function _existingBatch(uint256 batchId) private view returns (Batch storage b) { b = _batches[batchId]; if (b.status == BatchStatus.None) revert UnknownBatch(batchId); } } </pre>	
contracts/script/Deploy.s.sol	Solidity
<pre> // SPDX-License-Identifier: MIT pragma solidity ^0.8.24; import {Script, console} from "forge-std/Script.sol"; import {MedicineRegistry} from "../src/MedicineRegistry.sol"; contract Deploy is Script { function run() external returns (MedicineRegistry registry) { // The NMRA wallet: it becomes admin and regulator. address regulator = vm.envAddress("REGULATOR_ADDRESS"); vm.startBroadcast(); registry = new MedicineRegistry(regulator); vm.stopBroadcast(); console.log("MedicineRegistry deployed at:", address(registry)); } } </pre>	
contracts/slither.config.json	JSON
<pre> { "filter_paths": "node_modules lib test script" } </pre>	
contracts/.env.example	env
<pre> # Copy to contracts/.env (never commit .env) SEPOLIA_RPC_URL=https://eth-sepolia.g.alchemy.com/v2/<your-alchemy-key> ETHERSCAN_API_KEY=<your-etherscan-key> REGULATOR_ADDRESS=<NMRA MetaMask address> </pre>	

Appendix B Test files

Result: 20 tests passed (19 unit/attack/fuzz + 1 invariant suite with 3 invariants).

contracts/test/MedicineRegistry.t.sol

Solidity · Foundry test

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import {Test} from "forge-std/Test.sol";
import {MedicineRegistry} from "../src/MedicineRegistry.sol";

contract MedicineRegistryTest is Test {
    MedicineRegistry reg;

    address nmra = makeAddr("nmra");
    address maker = makeAddr("maker");
    address distributor = makeAddr("distributor");
    address pharmacyA = makeAddr("pharmacyA");
    address pharmacyB = makeAddr("pharmacyB");

    bytes16 constant SERIAL = bytes16(0x8f3a1c2b4d5e6f708192a3b4c5d6e7f8);

    function setUp() public {
        reg = new MedicineRegistry(nmra);
        vm.startPrank(nmra);
        reg.registerParticipant(maker, reg.MANUFACTURER_ROLE(), "CeyPharma Ltd");
        reg.registerParticipant(distributor, reg.DISTRIBUTOR_ROLE(), "Island Distributors");
        reg.registerParticipant(pharmacyA, reg.PHARMACY_ROLE(), "Pharmacy A, Galle");
        reg.registerParticipant(pharmacyB, reg.PHARMACY_ROLE(), "Pharmacy B, Matara");
        vm.stopPrank();
    }

    // ----- helpers

    function _register() internal returns (uint256 id) {
        bytes32[] memory hashes = new bytes32[](1);
        hashes[0] = reg.hashSerial(SERIAL);
        vm.prank(maker);
        id = reg.registerBatch(
            "Paracetamol 500mg", "PARA-2026-001", uint64(block.timestamp + 365 days), hashes
        );
    }

    function _readyAtPharmacyA() internal returns (uint256 id) {
        id = _register();
        vm.prank(nmra);
        reg.approveBatch(id);
        vm.prank(maker);
        reg.transferBatch(id, distributor);
        vm.prank(distributor);
        reg.transferBatch(id, pharmacyA);
    }

    function _verdict(bytes16 serial) internal view returns (MedicineRegistry.Verdict v) {
        (v,,, ) = reg.verify(reg.hashSerial(serial));
    }

    // ----- happy path

    function test_GenuinePackVerifies() public {
        _readyAtPharmacyA();
        assertEq(uint8(_verdict(SERIAL)), uint8(MedicineRegistry.Verdict.Genuine));
    }

    function test_CustodyTrailIsRecorded() public {
        uint256 id = _readyAtPharmacyA();
        MedicineRegistry.Handover[] memory trail = reg.getTrail(id);
        assertEq(trail.length, 2);
        assertEq(trail[0].to, distributor);
        assertEq(trail[1].to, pharmacyA);
    }
}
```

```

    assertEq(reg.getBatch(id).holder, pharmacyA);
}

function test_UnapprovedBatchShowsNotApproved() public {
    _register();
    assertEq(uint8(_verdict(SERIAL)), uint8(MedicineRegistry.Verdict.NotApproved));
}

function test_ExpiredPackShowsExpired() public {
    _readyAtPharmacyA();
    vm.warp(block.timestamp + 366 days);
    assertEq(uint8(_verdict(SERIAL)), uint8(MedicineRegistry.Verdict.Expired));
}

// ----- attack scenarios (demo slide)

function test_Attack_FakeCodeIsNotFound() public {
    _readyAtPharmacyA();
    assertEq(
        uint8(_verdict(bytes16(0xdeadbeefdeadbeefdeadbeefdeadbeef))),
        uint8(MedicineRegistry.Verdict.NotFound)
    );
}

function test_Attack_ClonedQrCodeIsFlaggedAfterSale() public {
    _readyAtPharmacyA();
    vm.prank(pharmacyA);
    reg.dispense(SERIAL);

    (MedicineRegistry.Verdict v,,, address soldBy,) = reg.verify(reg.hashSerial(SERIAL));
    assertEq(uint8(v), uint8(MedicineRegistry.Verdict.Dispensed));
    assertEq(soldBy, pharmacyA);
}

function test_Attack_CannotSellSamePackTwice() public {
    _readyAtPharmacyA();
    vm.prank(pharmacyA);
    reg.dispense(SERIAL);

    vm.prank(pharmacyA);
    vm.expectRevert(
        abi.encodeWithSelector(
            MedicineRegistry.AlreadyDispensed.selector, pharmacyA, uint64(block.timestamp)
        )
    );
    reg.dispense(SERIAL);
}

function test_Attack_RoguePharmacyCannotSellOthersStock() public {
    _readyAtPharmacyA();
    vm.prank(pharmacyB);
    vm.expectRevert(abi.encodeWithSelector(MedicineRegistry.NotHolder.selector, pharmacyA));
    reg.dispense(SERIAL);
}

function test_Attack_UnregisteredFactoryCannotRegisterBatch() public {
    address fakeFactory = makeAddr("fakeFactory");
    bytes32[] memory hashes = new bytes32[](1);
    hashes[0] = keccak256("fake");
    vm.prank(fakeFactory);
    vm.expectRevert(); // AccessControlUnauthorizedAccount
    reg.registerBatch("Fake Paracetamol", "FAKE-1", uint64(block.timestamp + 365 days), hashes);
}

function test_Attack_CannotHijackExistingSerial() public {
    _register();
    bytes32[] memory hashes = new bytes32[](1);
    hashes[0] = reg.hashSerial(SERIAL);
    vm.prank(maker);
    vm.expectRevert(

```

```

        abi.encodeWithSelector(MedicineRegistry.DuplicateSerial.selector, hashes[0])
    );
    reg.registerBatch("Copy", "COPY-1", uint64(block.timestamp + 365 days), hashes);
}

function test_Attack_RecalledBatchCannotBeSold() public {
    uint256 id = _readyAtPharmacyA();
    vm.prank(nmra);
    reg.recallBatch(id, "Failed quality test");

    assertEq(uint8(_verdict(SERIAL)), uint8(MedicineRegistry.Verdict.Recalled));
    vm.prank(pharmacyA);
    vm.expectRevert(
        abi.encodeWithSelector(
            MedicineRegistry.InvalidStatus.selector, MedicineRegistry.BatchStatus.Recalled
        )
    );
    reg.dispense(SERIAL);
}

function test_Attack_RevokedDistributorCannotShip() public {
    uint256 id = _register();
    vm.prank(nmra);
    reg.approveBatch(id);
    vm.prank(maker);
    reg.transferBatch(id, distributor);

    vm.startPrank(nmra);
    reg.revokeParticipant(distributor, reg.DISTRIBUTOR_ROLE());
    vm.stopPrank();

    vm.prank(distributor);
    vm.expectRevert(MedicineRegistry.NotAuthorized.selector);
    reg.transferBatch(id, pharmacyA);
}

function test_Attack_CannotShipToUnregisteredAddress() public {
    uint256 id = _register();
    vm.prank(nmra);
    reg.approveBatch(id);
    address stranger = makeAddr("stranger");
    vm.prank(maker);
    vm.expectRevert(
        abi.encodeWithSelector(MedicineRegistry.InvalidRecipient.selector, stranger)
    );
    reg.transferBatch(id, stranger);
}

// ----- input validation

function test_RevertWhen_BatchIsEmptyOrTooLarge() public {
    vm.startPrank(maker);
    vm.expectRevert(abi.encodeWithSelector(MedicineRegistry.InvalidUnitCount.selector, 0));
    reg.registerBatch("X", "X", uint64(block.timestamp + 1 days), new bytes32[](0));

    vm.expectRevert(abi.encodeWithSelector(MedicineRegistry.InvalidUnitCount.selector, 201));
    reg.registerBatch("X", "X", uint64(block.timestamp + 1 days), new bytes32[](201));
    vm.stopPrank();
}

function test_RevertWhen_ExpiryInPast() public {
    bytes32[] memory hashes = new bytes32[](1);
    hashes[0] = reg.hashSerial(SERIAL);
    vm.prank(maker);
    vm.expectRevert(MedicineRegistry.InvalidExpiry.selector);
    reg.registerBatch("X", "X", uint64(block.timestamp), hashes);
}

function test_RevertWhen_NonRegulatorApproves() public {
    uint256 id = _register();

```

```

        vm.prank(maker);
        vm.expectRevert();
        reg.approveBatch(id);
    }

    // ----- fuzz tests

    function testFuzz_OnlyManufacturersCanRegister(address caller) public {
        vm.assume(caller != maker);
        bytes32[] memory hashes = new bytes32[](1);
        hashes[0] = reg.hashSerial(SERIAL);
        vm.prank(caller);
        vm.expectRevert();
        reg.registerBatch("X", "X", uint64(block.timestamp + 1 days), hashes);
    }

    function testFuzz_RandomSerialIsNotFound(bytes16 randomSerial) public {
        vm.assume(randomSerial != SERIAL);
        _readyAtPharmacyA();
        assertEq(uint8(_verdict(randomSerial)), uint8(MedicineRegistry.Verdict.NotFound));
    }

    function testFuzz_NoPharmacyCanSellTwice(bytes16 serial) public {
        bytes32[] memory hashes = new bytes32[](1);
        hashes[0] = reg.hashSerial(serial);
        vm.prank(maker);
        uint256 id = reg.registerBatch("X", "X", uint64(block.timestamp + 1 days), hashes);
        vm.prank(nmra);
        reg.approveBatch(id);
        vm.prank(maker);
        reg.transferBatch(id, pharmacyA);

        vm.prank(pharmacyA);
        reg.dispense(serial);
        vm.prank(pharmacyA);
        vm.expectRevert();
        reg.dispense(serial);
    }
}

```

```

// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

import {Test} from "forge-std/Test.sol";
import {StdInvariant} from "forge-std/StdInvariant.sol";
import {MedicineRegistry} from "../src/MedicineRegistry.sol";

/// @dev The fuzzer calls these functions in random order with random inputs.
contract Handler is Test {
    MedicineRegistry public reg;
    address public pharmacy;
    bytes16[] public serials;

    uint256 public successfulSales;
    mapping(bytes16 => bool) public sold;
    uint256 public doubleSales; // must stay 0

    constructor(MedicineRegistry _reg, address _pharmacy, bytes16[] memory _serials) {
        reg = _reg;
        pharmacy = _pharmacy;
        serials = _serials;
    }

    function dispense(uint256 index) external {
        bytes16 s = serials[bound(index, 0, serials.length - 1)];
        vm.prank(pharmacy);
        try reg.dispense(s) {
            if (sold[s]) doubleSales++;
        }
    }
}

```

```

        sold[s] = true;
        successfulSales++;
    } catch {}
}

function dispenseRandom(bytes16 s) external {
    vm.prank(pharmacy);
    try reg.dispense(s) {
        if (sold[s]) doubleSales++;
        sold[s] = true;
        successfulSales++;
    } catch {}
}

}

contract MedicineRegistryInvariantTest is StdInvariant, Test {
    MedicineRegistry reg;
    Handler handler;
    uint256 batchId;

    function setUp() public {
        address nmra = makeAddr("nmra");
        address maker = makeAddr("maker");
        address pharmacy = makeAddr("pharmacy");

        reg = new MedicineRegistry(nmra);
        vm.startPrank(nmra);
        reg.registerParticipant(maker, reg.MANUFACTURER_ROLE(), "Maker");
        reg.registerParticipant(pharmacy, reg.PHARMACY_ROLE(), "Pharmacy");
        vm.stopPrank();

        bytes16[] memory serials = new bytes16[](20);
        bytes32[] memory hashes = new bytes32[](20);
        for (uint256 i; i < 20; ++i) {
            serials[i] = bytes16(keccak256(abi.encode("serial", i)));
            hashes[i] = reg.hashSerial(serials[i]);
        }
        vm.prank(maker);
        batchId = reg.registerBatch(
            "Amoxicillin 250mg", "AMOX-01", uint64(block.timestamp + 365 days), hashes
        );
        vm.prank(nmra);
        reg.approveBatch(batchId);
        vm.prank(maker);
        reg.transferBatch(batchId, pharmacy);

        handler = new Handler(reg, pharmacy, serials);
        targetContract(address(handler));
    }

    function invariant_NeverSellMoreThanRegistered() public view {
        MedicineRegistry.Batch memory b = reg.getBatch(batchId);
        assertLe(b.dispensedCount, b.unitCount);
    }

    function invariant_NoPackIsSoldTwice() public view {
        assertEq(handler.doubleSales(), 0);
    }

    function invariant_ContractCountMatchesRealSales() public view {
        assertEq(reg.getBatch(batchId).dispensedCount, handler.successfulSales());
    }
}

```

Appendix C Frontend files

Type-checked with TypeScript 5.9 and built with Next.js 16.3, wagmi 3.7 and viem 2.56. Generate `lib/registryAbi.ts` with the command in M1.2. Don't copy it by hand.

frontend/.env.example

env

```
# Copy to frontend/.env.local for local development
NEXT_PUBLIC_CHAIN_ID=31337
NEXT_PUBLIC_REGISTRY_ADDRESS=0x5FbDB2315678afecb367f032d93F642f64180aa3
NEXT_PUBLIC_RPC_URL=
```

frontend/lib/config.ts

TypeScript

```
import { createConfig, http } from "wagmi";
import { foundry, sepolia } from "wagmi/chains";
import { injected } from "wagmi/connectors";

// NEXT_PUBLIC_CHAIN_ID: "31337" = local Anvil chain, "11155111" = Sepolia testnet
export const chain = process.env.NEXT_PUBLIC_CHAIN_ID === "31337" ? foundry : sepolia;
export const registryAddress = process.env.NEXT_PUBLIC_REGISTRY_ADDRESS as `0x${string}`;

export const config = createConfig({
  chains: [sepolia, foundry],
  connectors: [injected()],
  transports: {
    [sepolia.id]: http(process.env.NEXT_PUBLIC_RPC_URL),
    [foundry.id]: http(), // defaults to http://127.0.0.1:8545
  },
  ssr: true,
});
```

frontend/lib/tx.ts

TypeScript

```
import { BaseError, ContractFunctionRevertedError } from "viem";

const MESSAGES: Record<string, string> = {
  AlreadyDispensed: "Clone alert: this pack was already sold. The code may have been copied.",
  NotHolder: "Your organisation doesn't hold this batch.",
  UnknownSerial: "This code isn't registered. The pack may be counterfeit.",
  InvalidStatus: "This batch is not approved, or has been recalled.",
  BatchExpired: "This batch has expired.",
  InvalidRecipient: "That address isn't a registered distributor or pharmacy.",
  DuplicateSerial: "One of these serials is already registered.",
  AccessControlUnauthorizedAccount: "Your wallet doesn't have permission for this action.",
};

// Turns a contract revert (e.g. AlreadyDispensed) into a sentence a user understands.
export function friendlyError(err: unknown): string {
  if (err instanceof BaseError) {
    const revert = err.walk((e) => e instanceof ContractFunctionRevertedError);
    if (revert instanceof ContractFunctionRevertedError) {
      const name = revert.data?.errorName ?? "";
      return MESSAGES[name] ?? `Transaction rejected: ${name}`;
    }
    return err.shortMessage;
  }
  return String(err);
}
```

```
"use client";

import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
import { useState } from "react";
import { WagmiProvider } from "wagmi";
import { config } from "@lib/config";

export function Providers({ children }: { children: React.ReactNode }) {
  const [queryClient] = useState(() => new QueryClient());
  return (
    <WagmiProvider config={config}>
      <QueryClientProvider client={queryClient}>{children}</QueryClientProvider>
    </WagmiProvider>
  );
}
```

```
import type { Metadata } from "next";
import { Geist, Geist_Mono } from "next/font/google";
import "./globals.css";
import { Providers } from "../providers";

const geistSans = Geist({
  variable: "--font-geist-sans",
  subsets: ["latin"],
});

const geistMono = Geist_Mono({
  variable: "--font-geist-mono",
  subsets: ["latin"],
});

export const metadata: Metadata = {
  title: "Counterfeit Medicine Checker",
  description: "Verify medicine packs on the Ethereum blockchain",
};

export default function RootLayout({ children }: LayoutProps<"/">) {
  return (
    <html
      lang="en"
      className={` ${geistSans.variable} ${geistMono.variable} h-full antialiased`}
    >
      <body className="min-h-full flex flex-col">
        <Providers>{children}</Providers>
      </body>
    </html>
  );
}
```

```
import { VerifyResult } from "@components/VerifyResult";

// The QR code on each pack opens: https://<our-site>/verify?s=0x<32 hex characters>
type Props = { searchParams: Promise<{ s?: string }> };

export default async function VerifyPage({ searchParams }: Props) {
  const { s } = await searchParams;
  const valid = typeof s === "string" && /^0x[0-9a-fA-F]{32}$/.test(s);

  return (
    <main className="mx-auto max-w-md p-6">
      <h1 className="text-2xl font-bold">Medicine check</h1>
      {valid ? <VerifyResult serial={s as `0x${string}`} /> : <p>Invalid or missing code.</p>}
    </main>
  );
}
```

```

"use client";

import { keccak256 } from "viem";
import { useReadContract } from "wagmi";
import { chain, registryAddress } from "@lib/config";
import { registryAbi } from "@lib/registryAbi";
import { OrgName } from "../OrgName";

// Order must match the Verdict enum in MedicineRegistry.sol
const VERDICTS = [
  { label: "NOT FOUND: likely counterfeit", color: "bg-red-600" },
  { label: "Not yet approved by NMRA", color: "bg-amber-500" },
  { label: "GENUINE", color: "bg-green-600" },
  { label: "ALREADY SOLD", color: "bg-amber-500" },
  { label: "RECALLED: do not use", color: "bg-red-600" },
  { label: "EXPIRED", color: "bg-red-600" },
] as const;

export function VerifyResult({ serial }: { serial: `0x${string}` }) {
  // Reads are free and need no wallet, so patients can verify without MetaMask.
  const { data, isLoading, error } = useReadContract({
    address: registryAddress,
    abi: registryAbi,
    functionName: "verify",
    args: [keccak256(serial)],
    chainId: chain.id,
  });

  if (isLoading) return <p>Checking the blockchain...</p>;
  if (error || !data) return <p>Could not reach the network. Try again.</p>;

  const [verdict, batchId, batch, dispensedBy, dispensedAt] = data;
  const v = VERDICTS[verdict];

  return (
    <div className="mt-4 space-y-3">
      <div className={` ${v.color} rounded p-4 text-xl font-bold text-white`} >{v.label}</div>
      {Number(batchId) > 0 && (
        <ul className="text-sm">
          <li>Medicine: {batch.drugName}</li>
          <li>Batch: {batch.batchNumber}</li>
          <li>Expires: {new Date(Number(batch.expiryDate) * 1000).toLocaleDateString()}</li>
          {Number(dispensedAt) > 0 && (
            <li>
              Sold by <OrgName address={dispensedBy} /> on{" "}
              {new Date(Number(dispensedAt) * 1000).toLocaleString()}. If you didn't buy it
              there, report this pack.
            </li>
          )}
        </ul>
      )}
    </div>
  );
}

```

```
"use client";

import { useReadContract } from "wagmi";
import { chain, registryAddress } from "@lib/config";
import { registryAbi } from "@lib/registryAbi";

// Shows "Pharmacy A, Galle" instead of 0x90F7...b906
export function OrgName({ address }: { address: `0x${string}` }) {
  const { data } = useReadContract({
    address: registryAddress,
    abi: registryAbi,
    functionName: "orgName",
    args: [address],
    chainId: chain.id,
  });
  return <>{data || address}</>;
}
```

```
"use client";

import { useConnect, useConnection, useDisconnect } from "wagmi";
import { injected } from "wagmi/connectors";

export function ConnectButton() {
  const { address, isConnected } = useConnection();
  const connect = useConnect();
  const disconnect = useDisconnect();

  if (isConnected) {
    return (
      <button onClick={() => disconnect.mutate()}>
        {address?.slice(0, 6)}...{address?.slice(-4)} (disconnect)
      </button>
    );
  }
  return (
    <button onClick={() => connect.mutate({ connector: injected() })}>
      Connect MetaMask
    </button>
  );
}
```

```
"use client";

import { useState } from "react";
import { simulateContract, waitForTransactionReceipt, writeContract } from "wagmi/actions";
import { chain, config, registryAddress } from "@lib/config";
import { registryAbi } from "@lib/registryAbi";
import { friendlyError } from "@lib/tx";

export function DispenseForm() {
  const [serial, setSerial] = useState("");
  const [status, setStatus] = useState("");

  async function onDispense() {
    setStatus("Waiting for MetaMask...");
    try {
      // 1. Simulate: fails fast with a readable reason (e.g. AlreadyDispensed)
      const { request } = await simulateContract(config, {
        address: registryAddress,
        abi: registryAbi,
        functionName: "dispense",
        args: [serial as `0x${string}`],
        chainId: chain.id,
      });
      // 2. Send, 3. wait for confirmation
      const hash = await writeContract(config, request);
      await waitForTransactionReceipt(config, { hash, chainId: chain.id });
      setStatus(`Sold. Transaction: ${hash}`);
    } catch (err) {
      setStatus(friendlyError(err));
    }
  }

  return (
    <div className="space-y-2">
      <input
        value={serial}
        onChange={(e) => setSerial(e.target.value.trim())}
        placeholder="0x... serial from QR"
      />
      <button onClick={onDispense}>Mark as sold</button>
      <p>{status}</p>
    </div>
  );
}
```

```

"use client";

import { QRCodeSVG } from "qrcode.react";
import { useState } from "react";
import { keccak256, toHex } from "viem";
import { simulateContract, waitForTransactionReceipt, writeContract } from "wagmi/actions";
import { chain, config, registryAddress } from "@lib/config";
import { registryAbi } from "@lib/registryAbi";
import { friendlyError } from "@lib/tx";

// Each pack gets a random 128-bit serial. Only its hash goes on-chain at registration.
function newSerial(): `0x${string}` {
  return toHex(crypto.getRandomValues(new Uint8Array(16)));
}

export function NewBatchForm() {
  const [serials, setSerials] = useState<`0x${string}`[]>([]);
  const [status, setStatus] = useState("");

  async function onRegister() {
    const fresh = Array.from({ length: 10 }, newSerial);
    const expiry = BigInt(Math.floor(Date.now() / 1000) + 365 * 24 * 3600);
    setStatus("Waiting for MetaMask...");
    try {
      const { request } = await simulateContract(config, {
        address: registryAddress,
        abi: registryAbi,
        functionName: "registerBatch",
        args: ["Paracetamol 500mg", "PARA-2026-001", expiry, fresh.map((s) => keccak256(s))],
        chainId: chain.id,
      });
      const hash = await writeContract(config, request);
      await waitForTransactionReceipt(config, { hash, chainId: chain.id });
      setSerials(fresh); // show QR codes only after the batch is on-chain
      setStatus("Batch registered. Print the QR codes and keep the serial list private.");
    } catch (err) {
      setStatus(friendlyError(err));
    }
  }

  return (
    <div>
      <button onClick={onRegister}>Register batch of 10 packs</button>
      <p>{status}</p>
      <div className="grid grid-cols-3 gap-4">
        {serials.map((s) => (
          <QRCodeSVG key={s} value={`${window.location.origin}/verify?s=${s}`} size={128} />
        ))}
      </div>
    </div>
  );
}

```

```

import { ConnectButton } from "@components/ConnectButton";
import { DispenseForm } from "@components/DispenseForm";

export default function PharmacyPage() {
  return (
    <main className="p-6">
      <ConnectButton />
      <DispenseForm />
    </main>
  );
}

```

Appendix D CI workflow and local demo script

.github/workflows/ci.yml

YAML · GitHub Actions

```
name: CI

on:
  push:
    branches: [main]
  pull_request:

jobs:
  contracts:
    runs-on: ubuntu-latest
    defaults:
      run:
        working-directory: contracts
    steps:
      - uses: actions/checkout@v7
        with:
          submodules: recursive
      - uses: actions/setup-node@v7
        with:
          node-version: 24
      - run: npm ci
      - uses: foundry-rs/foundry-toolchain@v1
      - run: forge fmt --check
      - run: forge build --sizes
      - run: forge test -vvv

  slither:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
        with:
          submodules: recursive
      - uses: actions/setup-node@v7
        with:
          node-version: 24
      - run: npm ci
        working-directory: contracts
      - uses: crytic/slither-action@v0.4.2
        with:
          target: contracts
          slither-config: contracts/slither.config.json
          fail-on: medium
```

The CI workflow hasn't been run on GitHub yet. Each step was run locally. If the Slither job gives trouble, remove it and run Slither locally instead.

```
#!/usr/bin/env bash
# Deploys to a fresh local Anvil chain and creates demo data.
# Run `anvil` in another terminal first. Anvil keys are public test keys: local use only.
set -e
RPC=http://127.0.0.1:8545
NMRA_KEY=0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbed5efcae784d7bf4f2ff80
MAKER_KEY=0x59c6995e998f97a5a0044966f0945389dc9e86dae88c7a8412f4603b6b78690d
DIST_KEY=0x5de4111afa1a4b94908f83103eb1f1706367c2e68ca870fc3fb9a804cdab365a
PHARM_KEY=0x7c852118294e51e653712a81e05800f419141751be58f605c371e15141b007a6

addr() { cast wallet address "$1"; }
send() { cast send "$REG" "$@" --rpc-url $RPC > /dev/null; }

REGULATOR_ADDRESS=$(addr $NMRA_KEY) forge script script/Deploy.s.sol \
  --rpc-url $RPC --private-key $NMRA_KEY --broadcast > /dev/null
REG=0x5FbDB2315678afecb367f032d93F642f64180aa3 # first deployment on a fresh Anvil
echo "Registry: $REG"

send "registerParticipant(address,bytes32,string)" $(addr $MAKER_KEY) \
  $(cast keccak MANUFACTURER_ROLE) "CeyPharma Ltd" --private-key $NMRA_KEY
send "registerParticipant(address,bytes32,string)" $(addr $DIST_KEY) \
  $(cast keccak DISTRIBUTOR_ROLE) "Island Distributors" --private-key $NMRA_KEY
send "registerParticipant(address,bytes32,string)" $(addr $PHARM_KEY) \
  $(cast keccak PHARMACY_ROLE) "Pharmacy A, Galle" --private-key $NMRA_KEY

S1=0x8f3a1c2b4d5e6f708192a3b4c5d6e7f8 # stays unsold
S2=0x1111222333444555666677778888 # gets sold
EXPIRY=$(( $(date +%s) + 365 * 24 * 3600 ))
send "registerBatch(string,string,uint64,bytes32[])" "Paracetamol 500mg" "PARA-2026-001" \
  $EXPIRY "[$(cast keccak $S1),$(cast keccak $S2)]" --private-key $MAKER_KEY
send "approveBatch(uint256)" 1 --private-key $NMRA_KEY
send "transferBatch(uint256,address)" 1 $(addr $DIST_KEY) --private-key $MAKER_KEY
send "transferBatch(uint256,address)" 1 $(addr $PHARM_KEY) --private-key $DIST_KEY
send "dispense(bytes16)" $S2 --private-key $PHARM_KEY

echo "Genuine: http://localhost:3000/verify?s=$S1"
echo "Sold:    http://localhost:3000/verify?s=$S2"
echo "Fake:    http://localhost:3000/verify?s=0xdeadbeefdeadbeefdeadbeefdeadbeef"
```